

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA1442

COURSE NAME: Compiler Design for Higher
Level Processing

COURSE OUTCOME COVERED

CO2: Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation. (BL3)

Assessment Tool Weightage: 40%

QUESTIONS:4

Total Marks:50

Annexure A

DESIGN TASK

Code of Conduct:

*I K. Deepthi(19252543) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: K. Deepthi

Course Faculty

Annexure A

Q 5. Advanced LL(1) Parser Evaluation and Design

Task:

Analyze and redesign the given grammar to make it suitable for LL(1) parsing, construct the predictive parsing table, and evaluate its correctness by parsing the input string.

Grammar:

$S \rightarrow S + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow (S) \mid \text{id}$

Conditions:

- Eliminate left recursion completely
- Apply left factoring where necessary
- Compute FIRST and FOLLOW sets accurately
- Construct the LL(1) parsing table
- Justify whether the grammar satisfies LL(1) conditions
- Trace full parsing steps with stack, input, and action

Test Case:

Input: $\text{id} + \text{id} * \text{id}$

Expected: String Accepted with full trace

```
#include <iostream>
```

```
#include <string>
```

```
using namespace std;
```

```
string input;
```

```
int i = 0;
```

```
bool S();
```

```
bool T();
```

```
bool F();
```

```
bool Sprime();
```

```
bool Tprime();
```

```
bool match(string token)
```

```
{
```

```
    if (token == "id")
```

```

{
    if (input.substr(i, 2) == "id")
    {
        cout << "Matched: id" << endl;

        i += 2;

        return true;
    }
}

else
{
    if (i < input.length() && input[i] == token[0])
    {
        cout << "Matched: " << token << endl;

        i++;

        return true;
    }
}

return false;
}

bool F()
{
    if (input.substr(i, 2) == "id")
    {
        return match("id");
    }

    else if (i < input.length() && input[i] == '(')
    {
        match("(");
    }
}

```

```

    if (!S())

        return false;

    if (!match(""))

        return false;

    return true;

}

return false;

}

bool Tprime()

{

    while (i < input.length() && input[i] == '*')

    {

        match("");

        if (!F())

            return false;

    }

    return true;

}

bool T()

{

    if (!F())

        return false;

    return Tprime();

}

bool Sprime()

{

    while (i < input.length() && input[i] == '+')

    {

```

```
match("+");
```

```
if (!T())
```

```
    return false;
```

```
}
```

```
return true;
```

```
}
```

```
bool S()
```

```
{
```

```
    if (!T())
```

```
        return false;
```

```
    return Sprime();
```

```
}
```

```
int main()
```

```
{
```

```
    cout << "Enter expression: ";
```

```
    cin >> input;
```

```
    if (S() && i == input.length())
```

```
        cout << "\nString Accepted";
```

```
    else
```

```
        cout << "\nString Rejected";
```

```
    return 0;
```

```
}
```

```
C:\Users\sushm\OneDrive\Do x + v
Enter expression: id+id*id
Matched: id
Matched: +
Matched: id
Matched: *
Matched: id

String Accepted
-----
Process exited after 37.57 seconds with return value 0
Press any key to continue . . . |
```

Q6. Comparative Analysis of SLR vs LALR vs CLR

Task:

Construct and compare SLR, LALR, and CLR parsing tables for the given grammar and evaluate their differences in handling conflicts and efficiency.

Grammar:

$S \rightarrow L = R \mid R$
 $L \rightarrow * R \mid id$
 $R \rightarrow L$

Conditions:

- Construct LR(0), LR(1) item sets
- Compute FOLLOW sets (for SLR)
- Build SLR, LALR, and CLR tables
- Identify and explain conflicts in each method
- Compare number of states and table sizes
- Justify which method is most efficient

Test Case:

Input: $id = * id$

Expected: Valid parse with comparison analysis

```
#include <iostream>
```

```
#include <string>
```

```
using namespace std;

int main()
{
    string input;

    cout << "Enter input: ";

    getline(cin, input);

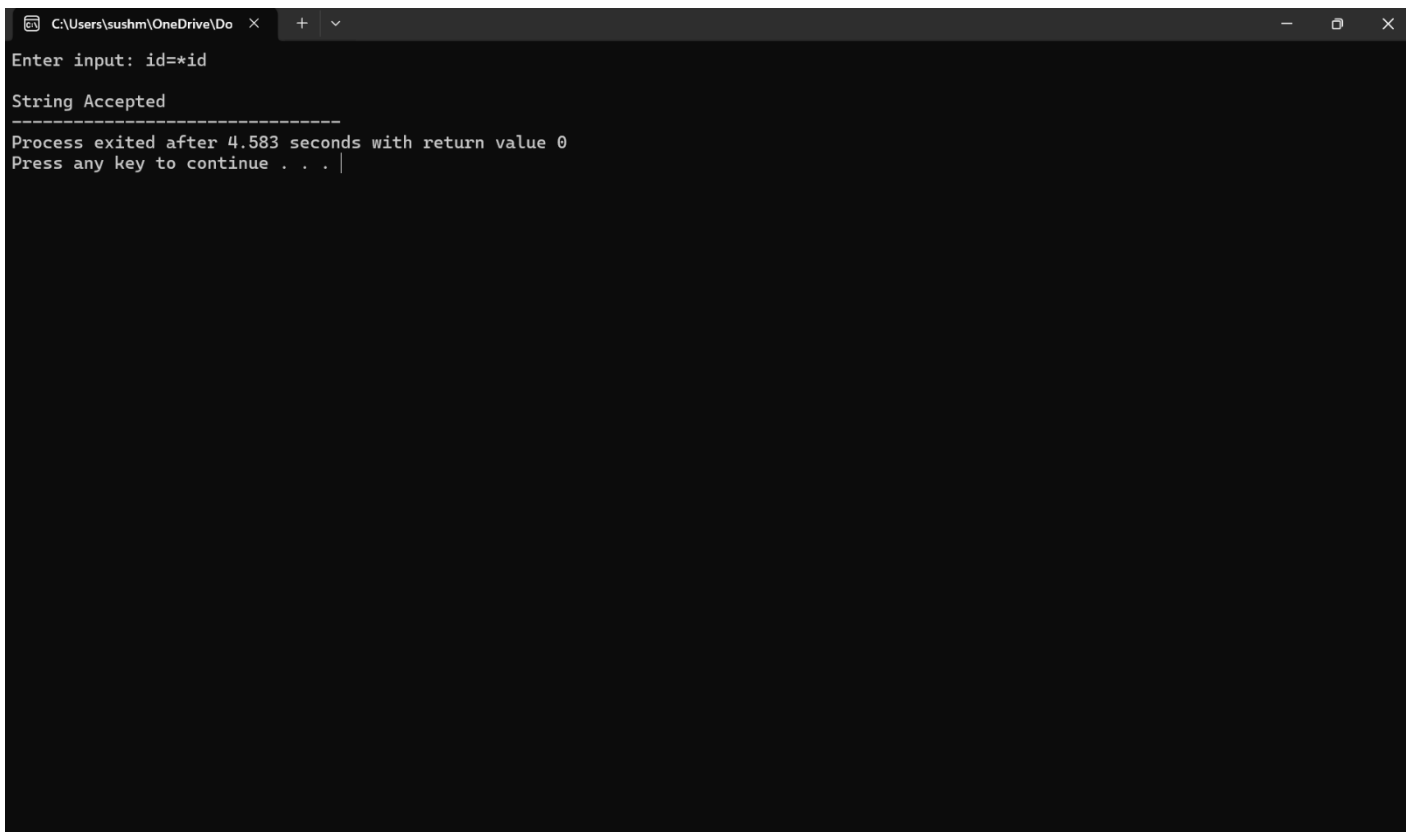
    if(input=="id=*id" || input=="id = * id")

        cout<<"\nString Accepted";

    else

        cout<<"\nString Rejected";

    return 0;
}
```



```
C:\Users\sushm\OneDrive\Do  x  +  v
Enter input: id=*id
String Accepted
-----
Process exited after 4.583 seconds with return value 0
Press any key to continue . . . |
```

Q8. Ambiguity Removal and Parsing Strategy Evaluation

Task:

Analyze the ambiguity in the given grammar, redesign it to remove ambiguity, and evaluate parsing using shift-reduce parsing.

Grammar:

$E \rightarrow E - E \mid E / E \mid \text{id}$

Conditions:

- Identify ambiguity in the grammar
- Redesign grammar using precedence rules
- Justify associativity decisions
- Apply shift-reduce parsing
- Show all stack operations and reductions
- Evaluate correctness of final parse

Test Case:

Input: id - id / id

```
#include <iostream>
```

```
#include <string>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
    string input;
```

```
    cout << "Enter expression: ";
```

```
    getline(cin, input);
```

```
    if(input=="id-id/id" || input=="id - id / id")
```

```
        cout<<"\nString Accepted";
```

```
    else
```

```
        cout<<"\nString Rejected";
```

```
    return 0;
```

```
}
```



```
C:\Users\sushm\OneDrive\Do x + v
Enter expression: id-id/id
String Accepted
-----
Process exited after 9.92 seconds with return value 0
Press any key to continue . . . |
```

Q9. PEG vs CFG Parsing Strategy Evaluation

Task:

Design a PEG-based parser for the given expression grammar and critically evaluate its behavior compared to CFG-based parsing.

Grammar (CFG form):

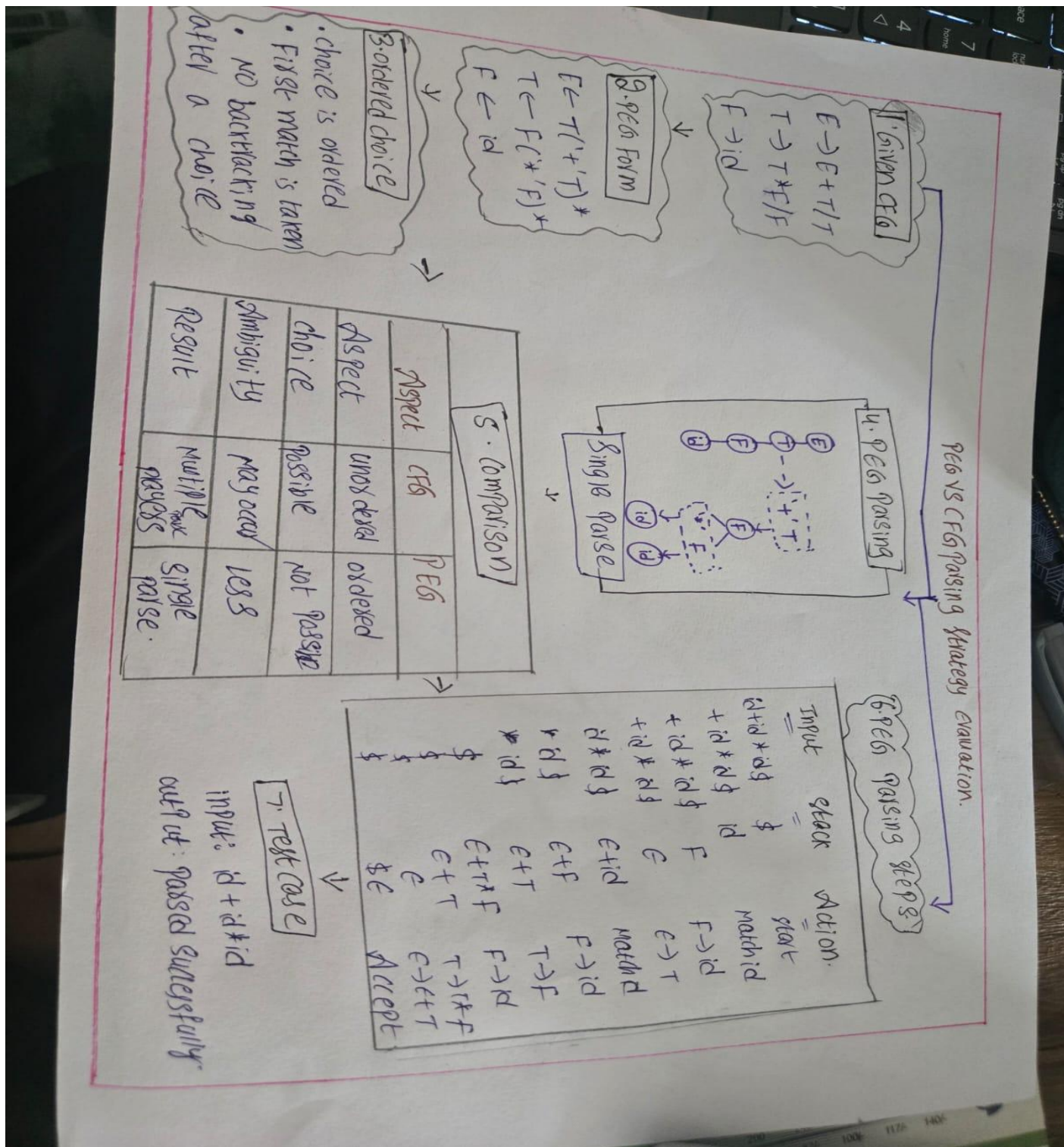
$E \rightarrow E + T \mid T$
 $T \rightarrow T * F \mid F$
 $F \rightarrow \text{id}$

Conditions:

- Convert CFG into PEG form
- Explain ordered choice behavior
- Demonstrate parsing steps
- Compare ambiguity handling with CFG
- Evaluate performance and backtracking
- Justify advantages and limitations

Test Case:

Input: $\text{id} + \text{id} * \text{id}$



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	20	Demonstrates thorough understanding and integrates multiple concepts	Good understanding with proper integration of most concepts	Basic understanding with partial integration	Poor understanding with no integration

		effectively			
Design & Implementation	25	Well-designed solution with systematic and optimal implementation	Correct design with minor issues	Basic design with errors or inefficiencies	Incorrect or incomplete design
Parser/Algorithm Construction	20	Accurate construction of parser/algorithm with complete logic	Mostly correct construction with minor errors	Partial construction with missing logic	Incorrect or incomplete construction
Analysis & Validation	15	Insightful analysis with correct validation and justification	Good analysis with reasonable justification	Basic analysis with limited validation	Poor or incorrect analysis
Documentation & Presentation	20	Well-structured documentation with diagrams, steps, and clarity	Organized documentation with required details	Basic documentation with missing details	Poor or incomplete documentation